MOD 4

Convolutional Neural Networks (CNNs) - Notes

1. Key Concepts

- **CNN Architecture** involves:
    - Input layer (e.g., RGB image)
    - Convolutional layers
    - ReLU Activation
    - Pooling layers (e.g., Max Pooling)
    - Fully Connected Layers (Dense)
    - Output layer

2. Convolution Operation

- Convolution is a filtering operation between input and a kernel/filter.
- A filter moves across the input (image), performing dot products to produce a feature map.
- Filters are also called kernels.
- Each filter produces **one output feature map**.
- Filters extend through the **depth** of the input.

3. Filter Dimensions

- A typical filter has:
    - Width × Height × Depth (same as input depth)
- Resultant output volume has:
    - Width × Height × Number of Filters

4. Hyperparameters in CNN

- **Filter Size**: Commonly 3×3, 5×5, etc.
- **Stride (S)**: How much the filter shifts at each step.
- **Padding**:
    - **Same Padding**: Output size = Input size
    - **Valid Padding**: No padding, output size shrinks
- **Depth**: Number of filters used

5. Pooling Layers

- Used for down-sampling.
- **Max Pooling** is common (e.g., 2×2 window, stride = 2).
- Reduces spatial size and computation, helps control overfitting.

6. Why Use CNNs?

- Patterns may be **small** and **repetitive** in the whole image.
- Shared weights reduce the number of parameters.
- CNNs efficiently capture **spatial hierarchies** (edges → shapes → objects).

7. Important Notes

- Filters **scan local regions** but capture global information by stacking layers.
- CNN can detect the same pattern even if it appears in **different regions** (translation invariance).
- Outputs from filters are stacked to form the **output volume**.

8. Example Question Prompts

- What does a 3D filter look like?
    - It spans the **depth** of the input (e.g., RGB → depth = 3).
- What does stride do?
    - Stride controls how much the filter moves at each step horizontally and vertically.

MOD 5

Recurrent Neural Networks (RNN)

Core Idea: Memory of past inputs via hidden states

Applications:

Language modeling

Speech recognition

Time-series forecasting

Problems with RNN

Vanishing/Exploding Gradients

Long Training Time

Difficulty with Long-term Dependencies

Ill-conditioning: Large sensitivity to input changes

Saddle Points: Flat gradient regions

Solutions

Proper weight initialization

Gradient clipping

Use LSTM/GRU

Regularization, dimensionality reduction

Learning rate tuning

Advanced RNN Architectures

LSTM (Long Short-Term Memory)

Input Gate, Forget Gate, Output Gate

Captures long-term dependencies

Applications: Translation, speech, forecasting

GRU (Gated Recurrent Unit)

Update Gate, Reset Gate

Simpler than LSTM, faster

Bidirectional RNN (Bi-RNN)

Uses past and future context

Echo State Networks (ESN)

Randomized reservoir

Only output weights trained

Good for temporal patterns

- **Meta Algorithms**: Algorithms that operate on other algorithms to improve performance or adaptivity.
- **Hyperparameter Tuning**: The process of selecting the best model configuration parameters to optimize performance.
- **Model Selection**: Choosing the best model among a set of candidates based on performance metrics.
- **Ensemble Methods**: Techniques that combine multiple models to produce a more accurate and robust prediction.
- **Cross-Validation**: A resampling method used to evaluate model performance by dividing data into training and validation sets multiple times.

Deep Learning Examination Answers

Q1 (Jan 2024 - Apr 2024)

i) **Challenges which have motivated Deep Learning:**

- High dimensional data
- Complex non-linear mappings
- Limited performance of traditional ML models
- Availability of computational power (GPUs)
- Data availability
- Automation in feature engineering

ii) **Probability Conditional Probabilities, Posterior Probability:**

- **Conditional Probability:** P(A|B) = P(A and B)/P(B)
- **Posterior Probability:** Updated probability after considering new evidence.
  Example: Diagnosing a disease based on symptoms.

iii) **Resampling Methods:**

- Cross-validation (k-fold, stratified)- used to estimate the test error associated with a model to evaluate its performance.
- Bootstrap sampling
- Leave-one-out cross-validation -is a better option than the validation set approach. Instead of splitting the entire dataset into two halves, only one observation is used for validation and the rest is used to fit the model.

iv) **Stochastic Gradient Descent:**

- Updates weights using one or few training samples
- Pros: Fast, efficient
- Cons: Noisy, may overshoot minima

v) **Activation Function:**

- Introduces non-linearity
- Common: ReLU, Sigmoid, Tanh
- ReLU: f(x) = max(0, x)

vi) **Performance Evaluation Parameters:**

- Accuracy
- Precision, Recall, F1-score

- Confusion matrix
- ROC-AUC score

Q2

i) **Hyperparameters:** Settings like learning rate, batch size, epochs

- **Parameter tuning:** Finding best values for parameters
- **Hyperparameter tuning:** Grid search, random search

ii) **Adaptive Learning Algorithms:**

- Adam, RMSProp, AdaGrad
- Adjust learning rate during training

Q2 A (Alt)

What is regularization?

- Prevents overfitting
- L1 (Lasso), L2 (Ridge)

Q2 B

i) **Loss Functions:**

- MSE (Regression): 1n∑(yi−y^i)2\frac{1}{n} \sum (y_i - \hat{y}_i)^2
- Cross Entropy (Classification): −∑ylog(y^)-\sum y \log(\hat{y})

ii) **Deep Network Architectures:**

- CNN: For spatial data like images

- RNN: Sequential data
- GAN: Generative tasks
- Autoencoders: Dimensionality reduction

Q3

i) **Input = 28x28x192, Kernel = 1x1, Filters = 32:**

- Output size = 28x28x32
- Effect: Reduces depth, retains spatial resolution, acts as dimensionality reducer

ii) Same input image: Answer remains 28x28x32

Q4

i) **CNN Architecture Parameters Calculation:**

Use:

- CONV: (F×F×D)×K+K(F \times F \times D) \times K + K
- FC: Input×Output+OutputInput \times Output + Output

Refer to architecture, calculate parameters for each layer.

ii) **Local Receptive Fields and Weight Sharing:**

- Local Receptive Field: Focused portion of input
- Weight Sharing: Same kernel weights across image

iii) **Valid vs Same Padding:**

- Valid: No padding, output smaller
- Same: Padding applied to maintain input size

- Variants: Full, Causal padding

Q5

Short Notes:

i) Hyperparameter Tuning

- Grid search, Random search

ii) Optimization with adaptive learning rates

- Adam, RMSProp

iii) Ill-conditioning and Saddle Points

- Ill-conditioning: Poor convergence
- Saddle Points: Flat regions hinder learning

iv) Deep Belief Networks (DBN)

- Layer-wise unsupervised pretraining of RBMs

v) Echo State Networks

- Recurrent architecture with fixed random weights in reservoir

vi) Bidirectional RNN

- Considers both past and future context

Q1 (Jan 2023 - May 2023)

i) **ReLU Activation Problems:**

- Dying ReLU

- Not learning if gradient = 0

ii) **Regularization:**

- Prevents overfitting
- L1, L2

iii) **CNN vs Dense Network:**

- CNN: Local connections, fewer parameters

iv) **Problems in RNN Training:**

- Vanishing/exploding gradients

v) **Bagging and Boosting:**

- Bagging: Parallel ensemble (e.g., Random Forest)
- Boosting: Sequential learning (e.g., AdaBoost)

vi) **Match the following:**
Done using mapping

Q2 A
i) Deep learning challenges:

- Vanishing gradient
- Overfitting
- Computation cost

ii) SGD:

- Pros: Efficient

- Cons: May fluctuate

Q2 B

i) Discriminative vs Generative Models:

- Discriminative: Learn decision boundary (e.g., Logistic Regression)
- Generative: Learn data distribution (e.g., GAN)

Q3

i) Deep architectures: CNN, RNN, DBN

ii) Activation Functions:

- ReLU, Leaky ReLU, ELU, Sigmoid, Tanh

Q4

i) CNN Parameter Calculation (Refer diagram):

- Use formula for CONV/FC layers as above

ii) Local Receptive Field, Weight Sharing

- Explained above

iii) Valid vs Same Padding

- Explained above

Q5

Write short notes:

i) Hyperparameter Tuning

ii) Optimization algorithms

iii) Ill-conditioning

iv) DBN

v) Echo State Networks

vi) Bidirectional RNN

Q1 (May 2024)
i) Convolution Operation Output (manual calculation)

ii) CNN Terms:
a. Transfer learning – using pretrained model
b. Padding – add borders to preserve dimensions
c. Pooling – downsampling

d. Flatten – convert 2D to 1D

e. Fully Connected Layer – every input connected to every output

iii) **Overfitting Techniques:**

- Regularization
- Dropout
- Data Augmentation
- Early Stopping

Q4
i) GRU vs LSTM:

- GRU: Fewer gates, simpler, faster
- LSTM: More control with input/output/forget gates

ii) RNN vs Feedforward:

- RNNs have memory (state)
- FFNNs have fixed input-output

iii) LSTM and Vanishing Gradient:

- LSTM uses gates to control flow
- Preserves gradient with cell state

Q5

Short Notes:

i) Unsupervised Pretrained Networks – DBNs

ii) Deep Belief Networks – RBMs stacked

iii) ReLU/Leaky ReLU – Activation to avoid vanishing gradient

iv) Variants of Basic Activation – ELU, Swish, GELU

v) Recursive Neural Networks – Tree structured

vi) Seq2Seq – Used in NLP, translation

—-------

---

1. Activation Function

- A function applied to the output of each neuron to introduce **non-linearity** into the model.
- Helps neural networks learn **complex patterns**.
- Common examples: **ReLU**, **Sigmoid**, **Tanh**.

---

2. Ill-conditioning and Saddle Points

- **Ill-conditioning**: When small input changes cause large output changes, making training **unstable or slow**.
- **Saddle point**: A point in the loss function where the gradient is zero but it's not a local minima/local maxima — can **trap the model** during optimization.

---

3. Adaptive Learning Rates

- Techniques that **automatically adjust the learning rate** during training.
- Help models converge **faster and more efficiently**.
- Examples: **Adam**, **RMSProp**, **Adagrad**.

---

4. Hyperparameter Tuning

- The process of finding the **best values** for model settings like learning rate, batch size, and number of layers.
- Methods: **Grid Search**, **Random Search**, **Bayesian Optimization**.

---

5. CNN (Convolutional Neural Network)

- A special type of neural network designed for **image and spatial data**.
- Uses **convolution layers** to extract features and **pooling layers** to reduce size.
- Followed by **fully connected layers** for final predictions.

---

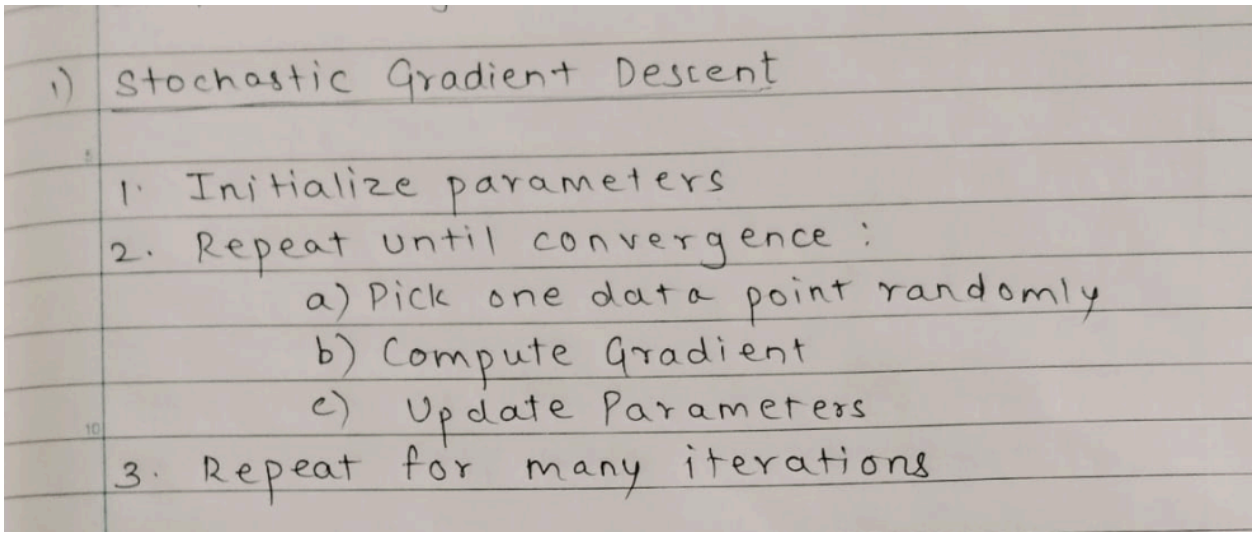
6. Regularisation

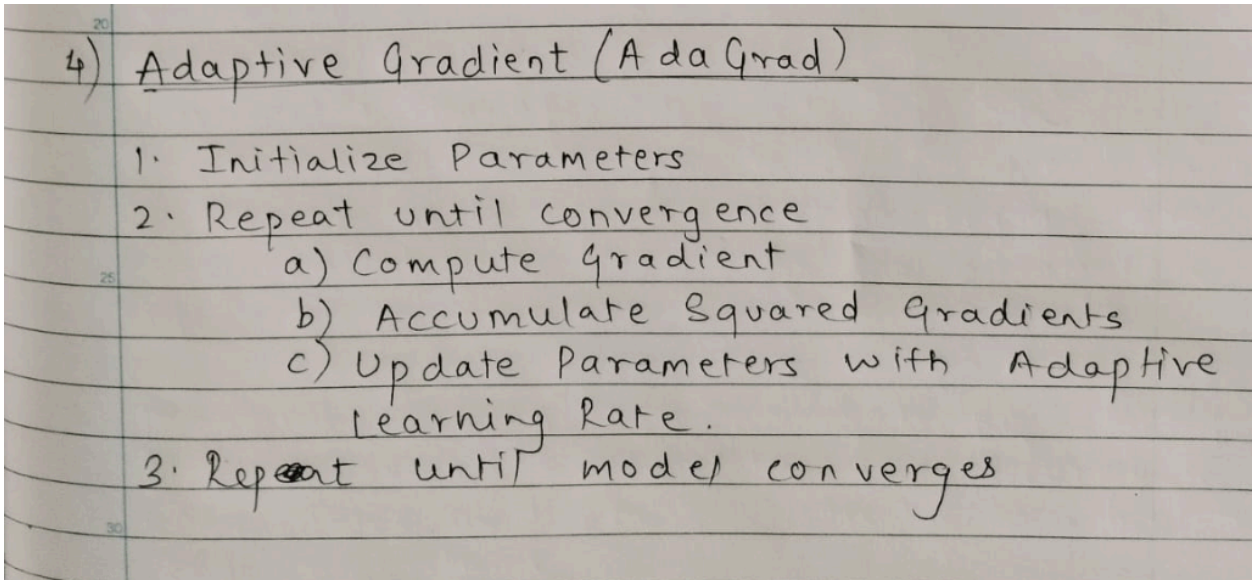
- Techniques to **prevent overfitting** (when a model performs well on training but poorly on test data).
- Common types:
    - **L1 (Lasso)**: Penalizes sum of absolute weights.
    - **L2 (Ridge)**: Penalizes sum of squared weights.
    - **Dropout**: Randomly disables neurons during training.

---

7. Gradients

- The **derivatives** of the loss function with respect to model parameters.
- Used in **backpropagation** to update weights and minimize loss.





---

8. **CNN Parameter Calculation**

a. **Convolution Layer Output Size**:

$$\text{Output size} = \left(\frac{\text{Input size} - \text{Filter size} + 2 \times \text{Padding}}{\text{Stride}} \right) + 1$$

b. **Parameters in Conv Layer**:

$$\text{Parameters} = (\text{Filter height} \times \text{Filter width} \times \text{Input channels} + 1) \times \text{Number of filters}$$

c. **Fully Connected (FC) Layer Parameters**:

$$\text{Parameters} = (\text{Input size} \times \text{Output size}) + \text{Output size}$$

One-line explanation: FC layers connect every input to every output neuron and are used for the **final classification or decision-making** in CNNs.

---

9. Valid and Same Padding

- **Valid Padding**: No padding; output size gets **smaller**.
- **Same Padding**: Adds padding so that output size stays **same** as input.
